

Procedural Content Generation in Virtual Environments using Interactive Genetic Algorithms

Anonymous Author(s)

Affiliation withheld for double-anonymous review

Abstract—Settlement generators in the IEEE Generative Design in Minecraft Competition (GDMC) are judged on four criteria—adaptability, functionality, aesthetics, and narrative—and aesthetics is the hardest to optimize because it resists formal definition. Rather than approximating human taste with hand-tuned proxies, we ask whether human taste can be measured directly. Our system combines a human-in-the-loop Interactive Genetic Algorithm (IGA) with Indirect Parametric Encoding, a box-split shape grammar, and a slope-aware ACO-Steiner road network, with the user serving as the fitness function during evolution. Because GDMC entries must be fully autonomous, we do not propose the IGA as a competition entry: we use it as a probe—a way to extract, in usable form, what evaluators actually respond to in procedurally generated settlements. A pilot study with five evaluators showed a small non-significant aesthetic advantage for IGA-generated cities (medium effect size), identified color as the dominant perceptual factor, and exposed evaluator fatigue as a measurable phenomenon already at this scale, while functionality and adaptability remained comparable across conditions. We discuss what these observations imply for the next, autonomous, GDMC-submittable iteration of the system.

Index Terms—Procedural Content Generation, Interactive Genetic Algorithm, Human-in-the-Loop, Minecraft, GDMC.

I. INTRODUCTION

A. Generative Design Minecraft Competition

The Generative Design Minecraft Competition (GDMC) is a yearly contest held during the IEEE Conference on Games. Recognizing the flaws in Minecraft’s own village generation system, it challenges participants to create a program that can build a settlement without human intervention. The main aspects that are evaluated are: adaptability, functionality, aesthetics and evocative narrative. All of these criteria are judged by a panel of experts in fields ranging from Minecraft modding and AI to architecture and city planning. These aspects are evaluated based on how convincingly the settlement could pass as human-made [1]. As an extra challenge, there is an optional category called Chronicle, which consists of generating the settlement’s story, both as a whole and for each of its inhabitants.

Most GDMC participants rely on template-based approaches combined with adaptive placement techniques, using pre-defined schematic structures alongside pathfinding algorithms such as A* and Minimum Spanning Tree [2]. Wave Function Collapse (WFC) has also been explored [3], although research has questioned its suitability for 3D Minecraft generation due to long execution times and frequent convergence failures [4].

Agent-based simulation has produced some of the most successful results in the competition. AgentCraft [5] grows settlements organically through agent interaction, while the entry from team *Mike’s Angels* [6], which went on to win GDMC 2022 edition, achieved the highest scores across all four evaluation categories using decentralized iterative planning. The growing prevalence of agent-based approaches is noted in a survey conducted by the contest organizers [2].

Evolutionary approaches to GDMC have been limited. Outside of competition specifically, Barthet, Liapis and Yannakakis [7] evolved Minecraft buildings using CPPNs and novelty search, representing a hybrid between evolutionary methods and neural representation learning. To the best of our knowledge, no prior GDMC entry has applied interactive evolutionary computation to settlement generation.

B. The Aesthetics Challenge

Out of the four criteria evaluated in GDMC, aesthetics presents a particular challenge. While adaptability or functionality can be modeled mathematically, aesthetics is inherently subjective and cannot be formally defined. Procedural Content Generation (PCG) methods can optimize measurable proxies such as color variance or structural symmetry, but fail to recognize the nuance human evaluators respond to. This is precisely where traditional PCG breaks down. Without a reliable aesthetics function to optimize, these methods cannot direct generation towards an aesthetically pleasing output consistently.

IGA addresses this directly by substituting human judgment for the fitness function [8], allowing subjective aesthetic criteria to drive the evolutionary search. The key challenge this introduces is user fatigue [9], the degradation of evaluation quality as the user is asked to make repeated judgments, which limits how many generations the algorithm can run before human input becomes unreliable.

C. Motivation

Despite the growing sophistication of PCG systems in GDMC, aesthetic quality remains an open challenge. The competition explicitly evaluates aesthetics as one of its four core criteria, but this dimension is inherently subjective and difficult to encode in a traditional fitness function. IGA offers a principled way to incorporate human aesthetic judgment into the generation process but has not been previously applied in this context. Since GDMC requires fully autonomous generation, a system relying on human input cannot be directly sub-

mitted to the competition. This work therefore treats GDMC as an experimental framework to investigate whether IGA can produce more aesthetically pleasing settlements, with the goal of informing future autonomous systems that capture learned aesthetic preferences.

D. Contributions

We make the following contributions to GDMC research:

- i) We introduce the first IGA-based system designed within the GDMC framework, combining an Interactive Genetic Algorithm (IGA), Indirect Parametric Encoding (IPE), a box-split shape grammar, and a slope-aware ACO-Steiner road network.
- ii) We report a pilot study with $N = 5$ evaluators that surfaces color dominance and evaluator-fatigue onset as critical factors that autonomous systems must account for.
- iii) We outline a research roadmap for distilling human-collected aesthetic preferences into autonomous fitness functions submittable to GDMC.

This roadmap clarifies how an interactive approach supports the competition’s objectives despite the lack of full autonomy. GDMC entries must be fully autonomous, which excludes IGA from being a direct competition submission. We argue, however, that human-in-the-loop systems can serve as research instruments for GDMC: by sampling human aesthetic preferences on settlements, an IGA characterizes what evaluators value and yields targets that autonomous systems can subsequently approximate. This frame of reference positions our work as preparatory research for autonomous GDMC entries rather than as a competing entry.

II. RELATED WORK

A. Procedural Generation Techniques

Interactive Genetic Algorithms (IGA) delegate fitness evaluation to human judgment, making them suitable for optimizing subjective criteria such as aesthetics [8]. User fatigue is a central challenge in IGA design, as the number of evaluations the user can perform is inherently limited [9]. IGAs have been successfully applied to PCG, with notable examples including Picbreeder [10] for image evolution and Galactic Arms Race [11] for weapon generation in games.

Shape grammars generate designs through iterative application of production rules on geometric shapes [12]. Reference [13] demonstrates the application of box-split grammars to building generation in Minecraft and serves as the basis for the shape grammar engine used in this work.

Indirect parametric encoding (IPE) represents solutions through a compact set of generative parameters rather than directly encoding each element of the solution, reducing search space and improving the heritability of successful traits [14]. This approach is particularly suited to material and aesthetic generation, where a low-dimensional genome can be decoded into high-dimensional outputs, such as coherent block palettes across a entire settlement, by projecting continuous parameters onto a discrete vocabulary of materials [15].

III. SYSTEM DESIGN

We have developed a system that uses IGA to generate settlements in Minecraft. This system is divided into different modules, each with its own responsibilities. The system takes as input the number of buildings in the city and the terrain in which it will be built, in the form of a *buildArea*. Figure 1 shows the architecture of the system.

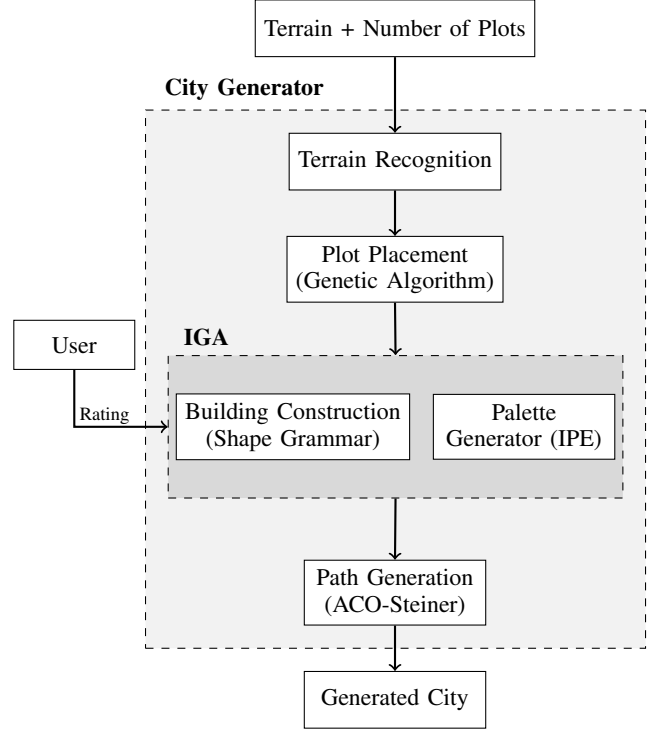


Fig. 1. Flowchart of the city generation system in *Minecraft*.

A. Terrain Recognition

The first module is responsible for inspecting the terrain. To do this, it makes use of GDPC’s *buildArea*. This allows both to get information about the blocks in the world as well as to place or remove blocks. In this stage, the module goes over the highest blocks in each of the coordinates of the building area to identify the orography. With this information, it builds three maps: one showing the types of blocks, one showing the height, and another one showing the slope. The slope of a given block is the sum of absolute differences between its height and the height of each adjacent block.

$$S_{x,y} = \sum_{i=-1}^1 \sum_{j=-1}^1 |h_{x,y} - h_{x+i,y+j}|, \quad (i,j) \neq (0,0) \quad (1)$$

B. Plot Allocation

After this, the next module to be executed is the one that allocates space of the plots into which the buildings will be built. To do this, each plot (as many as specified in the program’s input) is represented by its size, position, and type (either a single-family house or an apartment building). These

plots are placed by a genetic algorithm. Each individual from a given generation, i.e., a possible city, is represented by a list of plots. The death penalty [16] is applied to plots where more than 80% of the blocks are water. Otherwise, the fitness function penalizes terrain slope, water presence, and elongated plots; and rewards larger plot areas with more square shapes. The weighting constants control the relative influence of each term. The fitness function for a given city is the sum of the fitness of its plots and is minimized.

$$f(p) = \begin{cases} 10^{11}, & \text{if } B_W \geq 0.8A \\ \Phi, & \text{otherwise} \end{cases} \quad (2)$$

where Φ is defined as:

$$\Phi = \sigma S - \alpha A + \beta B_W + |h - w|$$

- **Area (A):** The area of the plot is subtracted, favoring larger plots.
- **Slope (S):** A penalty is applied based on terrain irregularity, calculated as the sum of height differences between all adjacent blocks within the plot. Using absolute values for these differences, the system ensures that local topographical irregularities contribute to the penalty even if the overall terrain is level, effectively guiding the genetic algorithm toward flatter surfaces for construction.
- **Water blocks (B_W):** The number of water blocks present in the plot.
- **Aspect Ratio (|h - w|):** A penalty derived from the difference between plot height (h) and width (w), favoring square-shaped plots over elongated ones.
- **The weighting parameters:** β , α , and σ are constant that allow control of the influence of each of the aspects that are penalized.

In addition to defining the dimensions and placement of the plots, this module also randomly assigns one of two uses:

- **loDesRes:** single-family houses, which may or may not have a yard depending on the size of the plot.
- **hiDesRes:** high-rise apartment buildings with multiple floors, each containing multiple apartments. Each apartment may or may not have a balcony and may be open or closed.

C. Building Construction

Now that all the plots have been defined, they have to be built on. For this, a combination of algorithms has been chosen, combining a Box-Split grammar [13] with an IGA [8], also using IPE [14] to combine them. Each plot will have what will be called a *block palette*. This block palette will contain the types of block that that plot will use for each functionality, e.g., floor, wall, door, etc. The most straightforward way to do this would be to simply associate a material to each functionality of each plot. Nevertheless, this direct mapping leads to a high-dimensional search space which could be too difficult for a Genetic Algorithm to navigate effectively. To address this, Indirect Parametric Encoding (IPE) has been chosen to codify the block palettes into each plot.

1) *Indirect Parametric Encoding:* To implement IPE, it is necessary to represent all available blocks in a common dimensional space. For this purpose, a database has been built collecting both aesthetic and semantic qualities of the materials. These values can either be continuous (e.g. color) or categories (e.g. type of material, like stone or wood), in which case one-hot encoding is used. To encode a property with one hot encoding, it is mapped to n dimensions, with n corresponding to the number of possible values that property can take. For each value, this n -dimensional vector will have a 1 on the value it represents and 0 elsewhere. This method ensures that all categories are equidistant and no unintended hierarchies arise between them. To simplify computational overhead and maintain a minimum level of coherence, the configuration vector is structured around a base vector (for the whole city), to which variations and offsets are added for each individual building. Following this mapping strategy, the configuration vector for the block palettes is defined as follows:

$$V_{\text{IPE}} = \{V_{\text{base}}, A, S, O, W\} \quad (3)$$

where:

a) *Base Vector* ($V_{\text{base}} \in \mathbb{R}^n$): Vector that defines the block palette in a latent space. This vector is in turn composed by the following components:

- **Color:** average color in HSVA format, 4 values normalized to the range [0,1].
- **Material:** one hot-encoding of the type of material (e.g. stone, wood, metal, etc.).
- **Category:** one-hot encoding of the "family" of blocks (e.g. sandstone, oak, copper, etc.). Works as a concretion of material.
- **Processing:** one-hot encoding of the level of processing of the block (e.g. raw, polished, chiseled, etc.).

For this implementation, the dimensionality n of the base vector is 51.

b) *Amplitude* ($A \in \mathbb{R}_+^n$): Amplitude vector that controls spatial variation per dimension. Each component $A_i \in [0.01, 0.5]$ determines how much can the value from the Base Vector at index i can fluctuate.

c) *Spatial Parameters* (S): These parameters control the spatial variation of the palette respective to the Base Vector, that is, how much the coordinates of each plot affect its deviation.

$$S = \{\text{frequency, seed, bias}\} \quad (4)$$

- **frequency** ≥ 0.1 : frequency of spatial patterns. Low values will generate smooth variations between nearby buildings, while high values will produce more abrupt changes.
- **seed** $\in \mathbb{N}$: seed for deterministic generation of procedural noise.
- **bias** $\in \mathbb{R}^n$, $|b_i| \leq 0.5$: offset per dimension that shifts the center of the distribution.

d) *Offsets per Slot (O)*:: Dictionary that associates each functional slot with an offset in the latent space:

$$O : \{\text{primary, accent, floor, ceiling, roof, } \dots\} \rightarrow \mathbb{R}^n \quad (5)$$

Each offset $O_{\text{slot}} \in [-0.5, 0.5]^n$ modifies the target vector for that specific slot, allowing to treat each functional slot (e.g. walls, ceilings, windows) differently.

e) *Distance weights (W $\in \mathbb{R}_+^4$)*:: Vector that assigns relative importance of each component when projecting from discrete to continuous space:

$$W = [w_{\text{color}}, w_{\text{material}}, w_{\text{categories}}, w_{\text{processing}}] \quad (6)$$

with $w_i \in [0.5, 3.0]$ for each i .

2) *Block Palette Generation*: For each functional space s within each building in coordinates (x, y) , the following implicit field function generates a target vector in the latent space:

$$v_{\text{obj}}(x, y, s) = V_{\text{base}} + \text{noise}(x, y, A) + \text{pattern}(x, y, S) + S_{\text{bias}} + O_s \quad (7)$$

where:

$$\text{noise}(x, y, A) = \mathcal{N}(0, A) \quad (\text{deterministic gaussian noise}) \quad (8)$$

$$\text{pattern}(x, y, S) = A \odot [\sin(x \cdot f), \cos(y \cdot f), \sin(r \cdot f/2)] \quad (9)$$

$$r = \sqrt{x^2 + y^2} \quad (10)$$

The target vector is projected onto the discrete block space through nearest neighbor search with weighted Euclidean distance:

$$\text{block}^* = \arg \min_{b \in C_s} d_W(v_{\text{obj}}, v_b) \quad (11)$$

where C_s is the set of valid blocks for slot s (determined by functional restrictions) and the weighted distance is defined as:

$$d_W(v_a, v_b) = \sqrt{\sum_{i=1}^4 w_i \cdot \|v_a^{(i)} - v_b^{(i)}\|_2^2} \quad (12)$$

with $v^{(i)}$ being the sub-vector corresponding to the component i (color, material, etc.).

3) *Shape Grammar*: Once all plots have their own block palette, a Box-Split Grammar will build all buildings in their corresponding plot. For this, the system developed by Eger [13] is used as a base, though with some slight modifications. Firstly, the rotation system has been overhauled, now allowing to rotate any of the boxes by 90, 180, or 270 degrees. To ensure the correct rotation of any structure, the rotation is applied not only to its respective scope, but also to all children of the scope to ensure nothing breaks. This allows us to build any structure oriented in any of the four cardinal directions without the need for defining additional rules.

Another addition to Eger's system is the implementation of logical operators in rule constraints. Originally, only one constraint could be applied per rule, and if it was not met, the rule would not execute. However, this proved to be insufficient in some rules where additional checks needed to be satisfied. For this reason, the logical operators *and* and *and or* were implemented, as well as comparators (*greater than*, *less than*, *equal than*, etc) in order to check restraints on a given scope dimensions.

One aspect present in Eger's original work that has proved vital is the probabilistic execution of rules. Whenever a rule is called, if there are two or more rules with that name and all of their constraints are fulfilled, the decision of which rule to call will be random. Moreover, different weights can be added to the rules to control the probabilities of execution of the rules.

The system has two separate grammars, one for each of the aforementioned types of building.

D. Interactive Genetic Algorithm

With the plots layout established and the shape grammar and IPE in place, the next thing to do is figuring out the most aesthetically pleasing configurations. This is the role of the IGA: users rate generated settlements, and this feedback drives the evolutionary search towards more aesthetically pleasing configurations.

The first thing to do when working with any genetic algorithm is the genome. In this case, the genome will be the vector of the IPE module, as well as eight weights ($[e_0, e_1, \dots, e_7]$, $e_i \geq 0$) to control probabilistic rules from the shape grammar. For each pair of weights, the probability of either option is its proportion in the sum of those two weights. These weights control the following aspects:

- $[e_0, e_1]$: in `loDesRes`, whether the house layout is square or L-shaped.
- $[e_2, e_3]$: in `hiDesRes`, whether an apartment has a terrace or a regular wall with windows.
- $[e_4, e_5]$: in `hiDesRes`, whether the terrace is open or enclosed.
- $[e_6, e_7]$: in `loDesRes`, whether the roof is flat or gabled.

As for the genetic operators, the system uses both crossover and mutation:

1) *Crossover*: a hybrid strategy is followed:

a) *Continuous Vectors*: Arithmetic crossover for V_{base} and W :

$$V_{\text{base}}^{\text{child}_1} = \frac{V_{\text{base}}^{G_1} + V_{\text{base}}^{G_2}}{2} \quad (13)$$

b) *Amplitudes, offsets per slot and grammar weights*::

Uniform crossover per dimension with probability $p = 0.5$.

c) *Spatial Parameters*: Complete selection of the spatial parameters of either of the parents.

d) *Distance Weights*: Arithmetic crossover.

2) *Mutation*: It applies random variations to the parameters with probability $p_{\text{mut}} = 0.2$, ensuring that the value always remains within the valid range.

a) *Continuous Vectors*: Gaussian mutation with $\sigma = 0.1$:

$$V'_{\text{base}} = V_{\text{base}} + \mathcal{N}(0, \sigma) \quad (14)$$

b) *Amplitudes*:

$$A'_i = A_i + \mathcal{N}(0, 0.05) \quad (15)$$

c) *Spatial Parameters*:

$$\text{frequency}' = \text{frequency} + \mathcal{N}(0; 0.1) \quad (16)$$

$$\text{seed}' = U(0, 10000) \quad (17)$$

$$\text{bias}'_i = \text{bias}_i + \mathcal{N}(0; 0.05) \quad (18)$$

d) *Offsets per Slot*:

$$O'_{s,i} = O_{s,i} + \mathcal{N}(0; 0.1) \quad (19)$$

e) *Distance Weights*:

$$w'_i = w_i + \mathcal{N}(0; 0.2) \quad (20)$$

f) *Grammar Weights*: discrete mutation with variation $\Delta \in [-1, 1]$, ensuring that the value is always greater than or equal to zero to ensure the proper functionality of the grammar.

3) *Interactive Evolutive Process*:: The algorithm employs a periodic fitness scheme in which the user acts as an implicit fitness function. The process is structured around cycles of length $\tau = 10$ generations. The algorithm is structured as follows:

Algorithm 1 Interactive Genetic Algorithm

```

1: Initialize population  $P_0$  with  $N = 10$  random genomes
2:  $g_{\text{winner}} \leftarrow \text{null}$ 
3: for  $t = 0$  to  $T_{\text{max}} = 200$  do
4:   if  $t \bmod \tau = 0$  then
5:     Select  $g_A, g_B \sim P_t$  {Random Uniform Prob.}
6:      $g_{\text{winner}} \leftarrow \text{USERTOURNAMENT}(g_A, g_B)$  {Feedback}
7:      $P_{t+1} \leftarrow \text{CHILDREN}(g_{\text{winner}}, N)$ 
8:   else if  $t \bmod \tau = 1$  then
9:      $P_t \leftarrow \text{INJECTDIVERSITY}(P_t, 2)$  {Contamination}
10:     $P_{t+1} \leftarrow \text{GENETICOPS}(P_t, g_{\text{winner}})$ 
11:   else
12:     $P_{t+1} \leftarrow \text{GENETICOPS}(P_t, g_{\text{winner}})$  {Local Search}
13:   end if
14: end for
15: return  $g_{\text{winner}}$ 

```

The algorithm employs a guided search strategy to balance preference-driven convergence with the maintenance of genetic diversity, effectively preventing population stagnation.

Phase 1 - Feedback (Gens. 0, 10, 20, ...): The user chooses one of two possible cities in a tournament. The winner g_{winner} becomes the center of exploration for the next $\tau - 1$ generations.

Phase 2 - Contamination (Gens. 1, 11, 21, ...): Two randomly-generated genomes are inserted into the population. This will add diversity in subsequent generations due to the genetic operations propagating these random variations across the entire population. This way, the algorithm does not converge early to local optima.

Phase 3 - Local Exploration (Gens. 2-9, 12-19, ...): Generations without feedback where the population evolves

by the way of the genetic operations. The winner genome from the last feedback is preserved (elitism), so as to not lose the best known solution.

The feedback process has been implemented in the form of a graphical user interface which allows the user to:

- Build and visualize either of the two candidate cities.
- Clean the area and build and visualize the other city.
- Switch between visualizing either city freely.
- Choose one of the cities.

Because the IGA is executed after the plot allocation, both cities are built over the same plot configuration, being the only change between them, the buildings constructed upon said plots.

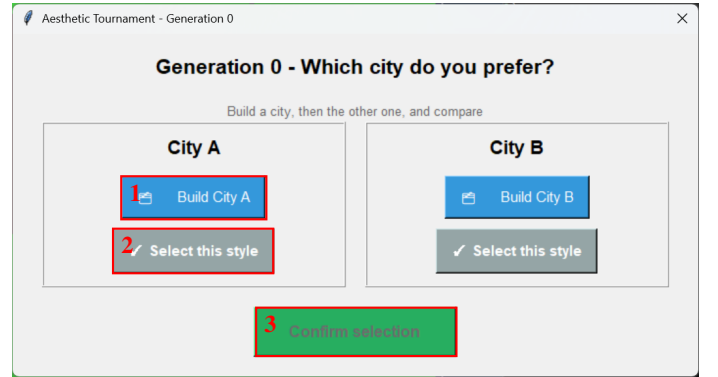


Fig. 2. Tournament GUI.

The numerated elements correspond to:

- 1) Button for building each of the cities. Before building any city, all plots are cleared out in order to delete any previous buildings that may have been built before.
- 2) Button for selecting either of the cities.
- 3) Button for submitting the selection and progressing to the next generation.

With this implementation, the IGA allows us to visualize and select iteratively the generated solutions, making sure that the aesthetic evolution aligns with the user preferences while maintaining genetic diversity.

E. Road Generation

Finally, the last module to be executed is the road module. A two-phase hybrid approach connects all buildings: ACO-Steiner [17] determines the network topology, and A* computes the specific routes. The ACO-Steiner cost function is modified to penalize elevation changes alongside horizontal distance:

$$C_{\text{total}} = \sum_{(u,v) \in E} [d_M(u, v) + k \cdot \Delta h(u, v)] \quad (21)$$

where d_M is the Manhattan distance and $\Delta h(u, v)$ is the absolute elevation difference between nodes. This guides the topology toward flatter routes without significant performance overhead. Once the topology is established, A* resolves each connection using a movement cost that incorporates

terrain slope, favoring flat surfaces and avoiding water. The coordinates occupied by the buildings are registered during construction and passed to A* as obstacles. Roads are rendered at a width of two blocks, and a post-processing pass places streetlights at regular intervals along the network.

IV. EXPERIMENT

To test the performance of the system, an experiment was carried out with five test users. In this experiment, each user generated a city using IGA. At the same time, 5 more cities were generated, but instead of the IGA the aesthetic vector was generated randomly. Then, all users rated the 5 random cities along with the ones the other users created (i.e. they did not rate their own city), not knowing which ones used the IGA and which ones were random. After the ratings, they were asked to fill a satisfaction survey about their experience with the system.

The experiment was carried out in a "Single Biome" world, with the biome "Plains" and the seed -6922832766660509343. The building area was set from coordinates XZ (0, 0) to (100, 100). Every city was generated under the same conditions.

The users were asked to rate three metrics: aesthetics, functionality and adaptability, as well as a global rating of the city. Although the IGA only optimizes for aesthetic quality, it is worth monitoring the other metrics in order to make sure that the IGA does not negatively impact them. This section will discuss as well the thoughts the users reflected on the satisfaction survey. For future reference, the cities generated randomly have ids 1 through five, while the ones generated by the users have ids 6 through 10.

It is worth noting that there were two problems during the experiment that should be taken into account when analyzing the results:

- During the creation of city 6 the communication between the program and Minecraft was lost, abruptly ending the construction of the city, specifically of one of the buildings, which ended up half-built. Nevertheless, since this interruption occurred right before the last generation, it was decided that the city would be rated along with the rest as usual.
- In city 7, due to a misunderstanding with one of the users, the user generated one random city and then built the city with the IGA right on top of the random city, causing the end city to be much more cluttered than a regular one with some generation errors (cut off buildings, roads leading to walls, etc.). This was noticed by the rest of users, who, on one hand, appreciated a greater density, but on the other hand, perceived the city as too haphazard.

V. RESULTS AND DISCUSSION

A. Results

Aesthetic ratings show a trend toward higher scores for IGA-generated cities (Fig. 4) compared to the random baseline (Fig. 3). As summarized in Table I, the IGA group obtained a mean rating 0.46 points higher than the random group. Welch's t-test yields $p = 0.42$, indicating that the difference

is not statistically significant given the small sample size. City 8 received the highest average aesthetic score, with users highlighting its color harmony, while city 1 was the most polarizing, with users divided over its use of contrasting colors.

TABLE I
AESTHETIC RATING SUMMARY BY GROUP

Group	Mean (μ)	Std. Dev. (σ)
Random	6.84	0.852
IGA	7.30	0.660
Welch's t-test: $p = 0.42$		

Functionality and adaptability ratings showed no systematic difference between the IGA and random groups. We frame this equivalence as a positive methodological outcome rather than a null finding: aesthetic optimization did not regress non-aesthetic criteria, a non-trivial property for hybrid PCG systems that risk over-specializing on the dimension they optimize for. Users evaluated functionality primarily based on building layout density and road network efficiency, noting occasional accessibility issues where building entrances were obstructed or not level with the ground. Adaptability ratings were the most inconsistent across users, reflecting a lack of consensus on what constitutes good terrain adaptation.

After the experiment, users were asked to fill in a satisfaction survey about their experience using the program. In it, they described the experience of using the IGA as positive, appreciating the coherence of the cities generated. However, they also expressed the wish to have a more explicit control over the evolution, such as the ability to influence building placement and also being able to isolate specific features they may like to ensure that they are not lost in the following generations.

Analyzing the time to choose a city for each generation of each user, the problem of user fatigue [9] has arisen. As the generations succeeded one another, the decision time of the users grew increasingly irregular. Some of the users were observed exploring the terrain and the structures leisurely, consistent with the loss of focus typical of user fatigue.

B. Interpretation

Beyond statistical inference, the magnitude of the observed aesthetic difference is informative on its own. With pooled standard deviation $\bar{\sigma} = \sqrt{(\sigma_{IGA}^2 + \sigma_{rand}^2)/2} \approx 0.76$, Cohen's $d = \Delta\mu/\bar{\sigma} \approx 0.60$, a medium effect by conventional thresholds. The non-significant p -value therefore reflects the limited power of a five-evaluator pilot rather than an absent effect, and the direction of the difference is consistent with the IGA's intended function. The dominance of color in evaluator judgment, observed consistently across all five debrief sessions, is the most actionable finding for autonomous GDMC entries: it identifies where to allocate latent-space capacity (more dimensions on color, fewer on category and processing) and suggests perceptual proxies that may approximate human aesthetic preference without a runtime evaluator. This convergence is consistent with formal treatments of game aesthetics

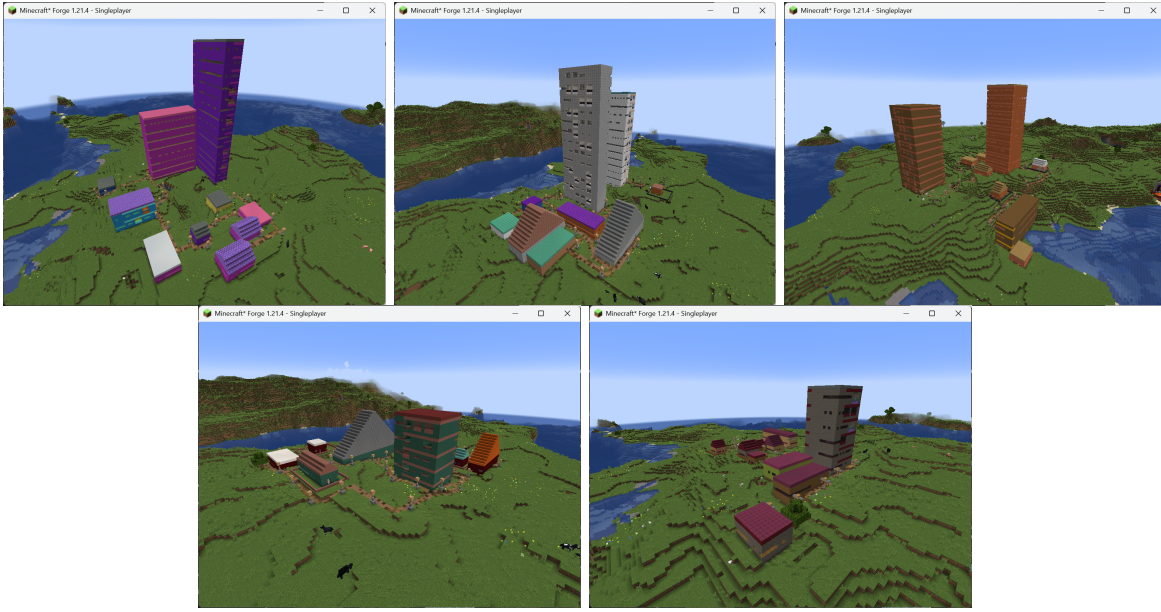


Fig. 3. Cities 1-5, randomly generated.

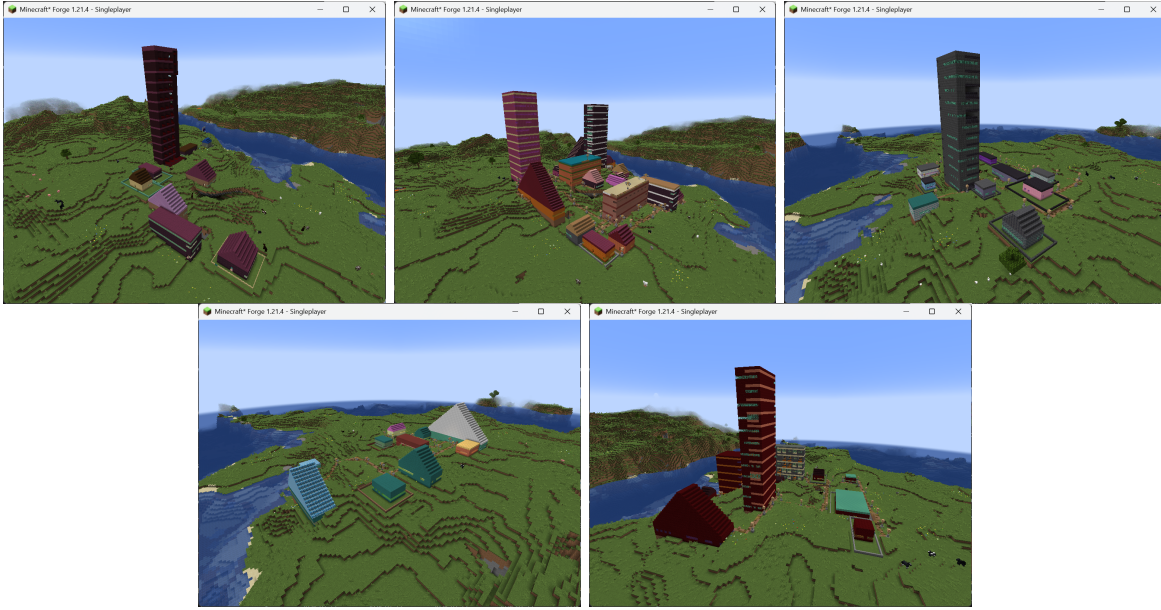


Fig. 4. Cities 6-10, generated by users. Note the problems described in cities 6 and 7.

both as one of the three pillars of the MDA framework [18] and as a primary perceptual lens in game design practice [19].

C. Limitations

Several aspects of this pilot deserve explicit framing for replication. The sample of five evaluators prevents inferential testing and renders all effect estimates hypothesis-generating rather than confirmatory. Two procedural incidents—the abrupt termination of city 6 due to a Minecraft-program disconnection, and the unintended overlap of a random and an IGA generation in city 7—were retained transparently in the dataset but plausibly bias the ratings of those two cities. The

single-biome, single-seed setup limits external validity to other terrains, and the exclusion of self-rating reduced bias at the cost of data volume. The color dominance finding relies on qualitative debrief reports rather than a factorial design that would isolate color from confounded IPE dimensions, and the order of rating presentation was not counterbalanced between users.

VI. CONCLUSION AND FUTURE WORK

We presented an interactive system that combines an Interactive Genetic Algorithm with Indirect Parametric Encoding, a box-split shape grammar, and a slope-aware ACO-Steiner

road network to generate Minecraft settlements within the GDMC framework. A pilot user study found a small, non-significant aesthetic advantage for IGA-generated cities, while functionality and adaptability remained comparable across conditions. Color emerged as the dominant perceptual factor in evaluator judgment, and evaluator fatigue became measurable already at this scale. Together, these findings position interactive evolution as a preference-probing instrument for GDMC: rather than competing directly, the IGA produces the data that autonomous systems require to internalize human aesthetic judgment at scale.

The natural next step is a fully autonomous descendant of the present system, eligible for direct submission to a future GDMC edition. The pilot data collected here can be used to train a learned surrogate fitness function on settlement rating pairs, following the surrogate fitness paradigm of Yannakakis and Togelius [20] and Hastings et al. [11]; such a surrogate would replace the human in the loop while approximating the expert-panel judgment required by the competition. The observed dominance of color further suggests biasing the autonomous palette generator toward perceptually salient configurations within the IPE latent space, reducing latent dimensionality along this axis to narrow the search. In parallel, the fatigue patterns characterized here can inform sampling strategies and rating-consolidation methods for any synthetic evaluator trained on subsequent rounds of human input.

ACKNOWLEDGMENTS

Redacted for double-anonymous review.

IA disclosure: ChatGPT 5.5 has been used for english proofreading.

REFERENCES

- [1] C. Salge, M. C. Green, R. Canaan, and J. Togelius, "Generative design in Minecraft (GDMC): Settlement generation competition," in *Proc. 13th Int. Conf. on the Foundations of Digital Games (FDG)*, Aug. 2018, pp. 1–10.
- [2] C. Salge, M. C. Green, R. Canaan, F. Skwarski, R. Fritsch, A. Brightmoore, S. Ye, C. Cao, and J. Togelius, "Impressions of the GDMC AI settlement generation challenge in Minecraft," in *Proc. 17th Int. Conf. on the Foundations of Digital Games (FDG)*, Sep. 2022, pp. 1–16.
- [3] ScholliYT, "MGAIA-Minecraft-GDMC: Procedural content generation in Minecraft for the GDMC competition," GitHub repository, 2022. [Online]. Available: <https://github.com/ScholliYT/MGAIA-Minecraft-GDMC4>
- [4] G. G. Manners, "Procedural content generation in 3 dimensions using wave function collapse in Minecraft," Honor's thesis, Trinity University, 2023. [Online]. Available: https://digitalcommons.trinity.edu/compsci_honors/69
- [5] A. Iramanesh and M. Kreminski, "AgentCraft: An agent-based Minecraft settlement generator," University of California, Santa Cruz, Tech. Rep., 2021. [Online]. Available: <https://escholarship.org/uc/item/83p272pq>
- [6] A. van der Staaij, J. Prins, V. Prins, J. Poelsma, L. Smit, M. Müller-Brockhausen, and M. Preuss, "Believable Minecraft settlements by means of decentralised iterative planning," in *Proc. IEEE Conf. on Games (CoG)*, Boston, MA, USA, 2023, pp. 1–8.
- [7] M. Barthet, A. Liapis, and G. N. Yannakakis, "Open-ended evolution for Minecraft building generation," *IEEE Transactions on Games*, vol. 15, no. 4, pp. 603–612, Jul. 2022.
- [8] H. Takagi, "Interactive evolutionary computation: Fusion of the capabilities of EC optimization and human evaluation," *Proceedings of the IEEE*, vol. 89, no. 9, pp. 1275–1296, 2001.
- [9] X. Llorà, K. Sastry, D. E. Goldberg, A. Gupta, and L. Lakshmi, "Combating user fatigue in IGAs: Partial ordering, support vector machines, and synthetic fitness," in *Proc. 7th Annual Conf. on Genetic and Evolutionary Computation (GECCO)*, Jun. 2005, pp. 1363–1370.
- [10] J. Secretan, N. Beato, D. B. D'Ambrosio, A. Rodriguez, A. Campbell, and K. O. Stanley, "Picbreeder: Evolving pictures collaboratively online," in *Proc. SIGCHI Conf. on Human Factors in Computing Systems (CHI)*, Apr. 2008, pp. 1759–1768.
- [11] E. J. Hastings, R. K. Guha, and K. O. Stanley, "Evolving content in the Galactic Arms Race video game," in *Proc. IEEE Symposium on Computational Intelligence and Games (CIG)*, Sep. 2009, pp. 241–248.
- [12] G. Stiny, "Introduction to shape and shape grammars," *Environment and Planning B: Planning and Design*, vol. 7, no. 3, pp. 343–351, 1980.
- [13] M. Eger, "Instant architecture in Minecraft using box-split grammars," in *Proc. 17th Int. Conf. on the Foundations of Digital Games (FDG)*, Sep. 2022, pp. 1–9.
- [14] J. Clune, K. O. Stanley, R. T. Pennock, and C. Ofria, "On the performance of indirect encoding across the continuum of regularity," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 3, pp. 346–367, Jun. 2011.
- [15] G. S. Hornby and J. B. Pollack, "Body-brain co-evolution using L-systems as a generative encoding," in *Proc. 3rd Annual Conf. on Genetic and Evolutionary Computation (GECCO)*, San Francisco, CA, USA, 2001, pp. 868–875.
- [16] Ö. Yeniyay, "Penalty function methods for constrained optimization with genetic algorithms," *Mathematical and Computational Applications*, vol. 10, no. 1, pp. 45–56, Apr. 2005.
- [17] Y. Hu, T. Jing, Z. Feng, X.-L. Hong, X.-D. Hu, and G. Yan, "ACO-Steiner: Ant colony optimization based rectilinear Steiner minimal tree algorithm," *Journal of Computer Science and Technology*, vol. 21, no. 1, pp. 147–152, Jan. 2006.
- [18] R. Hunicke, M. LeBlanc, and R. Zubeck, "MDA: A formal approach to game design and game research," in *Proc. AAAI Workshop on Challenges in Game AI*, 2004.
- [19] J. Schell, *The Art of Game Design: A Book of Lenses*. Morgan Kaufmann, 2008.
- [20] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*, 2nd ed. Cham, Switzerland: Springer, 2025.